Cyberscope

A TAC Security Company

Audit Report

VaultedMetrics Vesting

February 2026

SHA256 :

4e5798571e40f828789ee7643acf563fa15f2b5a47c6ebe727bce49097a8b1d3

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	16 Feb 2026

Source Files

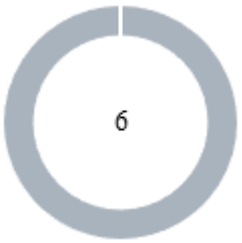
Filename	SHA256
VM888VestingCyberScope.sol	4e5798571e40f828789ee7643acf563fa15f2b5a47c6ebe727bce49097a8b1d3

Overview

VM888VestingCyberScope.sol

Vaulted Metrics Vesting implements a linear vesting schedule for three hardcoded team allocations totaling 66,600,000 VM888: Founder (70%), VP (20%), and a Future Reserve (10%) whose address is set post-deployment. The contract must be pre-funded before `startVesting()` initializes the vesting clock. Vesting accrues linearly over 5 years from `vestingStart`, with a 12-month cliff preventing any claims during the first year. The `claim()` function transfers the entire claimable balance in a single transaction — partial claims are not supported. Before executing, `claim()` verifies three conditions: the cliff has elapsed, unclaimed vested tokens exist, and the current spot price (read from the token contract) equals or exceeds 120% of the 30-day EMA, preventing team sells during downtrends. The contract uses a custom ownership model with `renounceOwnership()` for full autonomy, and an emergency `recoverERC20()` that explicitly excludes VM888 to protect beneficiary allocations.

Findings Breakdown



Critical	0
Medium	0
Minor / Informative	6

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	0	6	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	PAL	Potential Allocation Lock	Acknowledged
●	L03	Redundant Statements	Acknowledged
●	AME	Address Manipulation Exploit	Acknowledged
●	PTAI	Potential Transfer Amount Inconsistency	Acknowledged
●	UPR	Unprotected Price Reading	Acknowledged
●	L19	Stable Compiler Version	Acknowledged

PAL - Potential Allocation Lock

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L328
Status	Acknowledged

Description

The `setFutureReserveAddress` function does not verify whether the provided address is already assigned as an existing beneficiary. Specifically, the function lacks a check to ensure that `_address` is not equal to `FOUNDER_ADDRESS` or `VP_ADDRESS`. If the owner sets the future reserve address to an already registered beneficiary, the future reserve allocation would become permanently unclaimable, as the `_getAllocation` function would always return the original beneficiary's allocation before reaching the future reserve check.

```
Shell
function setFutureReserveAddress(address _address)
external onlyOwner {
    if (_address == address(0)) revert ZeroAddress();
    if (futureReserveAddress != address(0)) revert
FutureReserveAlreadySet();
    futureReserveAddress = _address;
    emit FutureReserveAddressSet(_address);

}
```

Recommendation

The team is advised to implement additional checks within the `setFutureReserveAddress` function to ensure that the provided address does not match any existing beneficiary address. This will prevent the future reserve allocation from becoming permanently locked.

Team Update

The team has acknowledged that this is not a security issue.

L03 - Redundant Statements

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L49
Status	Acknowledged

Description

Redundant statements are statements that are unnecessary or have no effect on the contract's behavior. These can include declarations of variables or functions that are not used, or assignments to variables that are never used.

As a result, it can make the contract's code harder to read and maintain, and can also increase the contract's size and gas consumption, potentially making it more expensive to deploy and execute.

```shell
error OwnershipRenounced();
```

Recommendation

To avoid redundant statements, it's important to carefully review the contract's code and remove any statements that are unnecessary or not used. This can help to improve the clarity and efficiency of the contract's code.

By removing unnecessary or redundant statements from the contract's code, the clarity and efficiency of the contract will be improved. Additionally, the size and gas consumption will be reduced.

Team Update

The team has acknowledged that this is not a security issue.

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L352
Status	Acknowledged

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

Shell

```
function recoverERC20(address tokenAddress, uint256 amount)
external onlyOwner {
    require(tokenAddress != address(vm888), "Cannot recover
VM888");
    (bool success, bytes memory data) = tokenAddress.call(
        abi.encodeWithSelector(0xa9059cbb, msg.sender,
amount)
    );
    require(success && (data.length == 0 ||
abi.decode(data, (bool))), "Transfer failed");

}
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

PTAI - Potential Transfer Amount Inconsistency

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L231
Status	Acknowledged

Description

The `claim` function updates the `totalClaimed` state variable with the expected transfer amount. However, the VM888 token implements a fee mechanism on transfers. If the vesting contract is not excluded from tax, the actual amount received by the beneficiary could be less than the recorded amount. As a result, there will be an inconsistency between the tracked claimed amount and the actual amount received by the beneficiary.

```
Shell
totalClaimed[msg.sender] += amount;

if (vm888.balanceOf(address(this)) < amount) revert
InsufficientContractBalance();


bool success = vm888.transfer(msg.sender, amount);
```

Recommendation

The team is advised to ensure that the vesting contract address is excluded from the token's tax mechanism. This will guarantee that the full expected amount is received by the beneficiary during claims, maintaining consistency between the tracked and the actual transferred amount.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

UPR - Unprotected Price Reading

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L220
Status	Acknowledged

Description

The vesting contract's claim() function verifies that the current spot price exceeds 120% of the 30-day EMA before allowing beneficiaries to withdraw their vested tokens. This price check is intended to prevent team members from claiming during market downtrends. However, both getCurrentPrice() and getEMA() are derived from the Uniswap V2 pair reserves at the moment of the transaction, reflecting instantaneous pool state rather than a time-averaged or manipulation-resistant price. As a result, a beneficiary can temporarily inflate the pool price within a single transaction to satisfy the price condition, bypassing the only protection that prevents token claims during unfavorable market conditions.

```shell
uint256 currentPrice = vm888.getCurrentPrice();
uint256 ema = vm888.getEMA();

if (ema > 0) {
    uint256 requiredPrice = (ema *
PRICE_THRESHOLD_MULTIPLIER) / 1e18;

    if (currentPrice < requiredPrice) revert
PriceConditionNotMet();
```

Recommendation

Price-dependent protocol decisions such as vesting claim gates should not rely on instantaneous spot prices derived from liquidity pool reserves. The team is advised to incorporate a price reading mechanism that is resistant to single-block manipulation, ensuring that the downtrend protection cannot be circumvented through temporary pool state distortion.

Team Update

The team has acknowledged that this is not a security issue and states:

The finding is either resolved by our ownership renouncement plan, mitigated by our deployment procedures, or represents an accepted design tradeoff that has been documented internally.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	VM888VestingCyberScope.sol#L2
Status	Acknowledged

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```shell
pragma solidity ^0.8.20;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Team Update

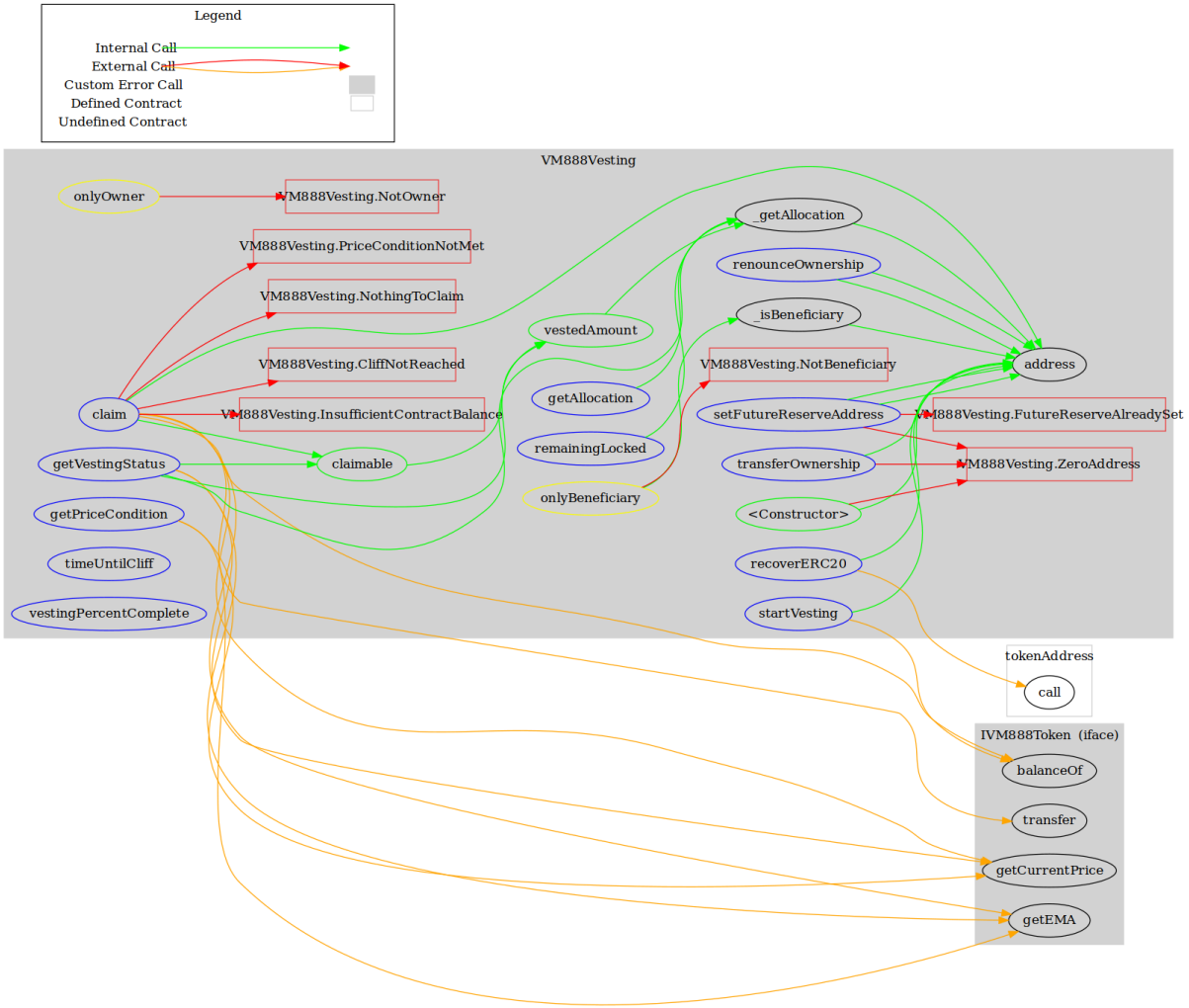
The team has acknowledged that this is not a security issue.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
VM888Vesting	Implementation			
		Public	✓	-
	startVesting	External	✓	onlyOwner
	vestedAmount	Public		-
	claimable	Public		-
	claim	External	✓	onlyBeneficiary
	getAllocation	External		-
	remainingLocked	External		-
	getPriceCondition	External		-
	getVestingStatus	External		-
	timeUntilCliff	External		-
	vestingPercentComplete	External		-
	setFutureReserveAddress	External	✓	onlyOwner
	transferOwnership	External	✓	onlyOwner
	renounceOwnership	External	✓	onlyOwner
	recoverERC20	External	✓	onlyOwner
	_getAllocation	Private		
	_isBeneficiary	Private		

Inheritance Graph

Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
VM888Vesting
onlyOwner
VM888Vesting.NotOwner
VM888Vesting.PriceConditionNotMet
VM888Vesting.NothingToClaim
VM888Vesting.CliffNotReached
VM888Vesting.InsufficientContractBalance
claim
getVestingStatus
getPriceCondition
timeUntilCliff
vestingPercentComplete
claimable
vestedAmount
getAllocation
remainingLocked
onlyBeneficiary
getAllocation
renounceOwnership
_isBeneficiary
VM888Vesting.NotBeneficiary
setFutureReserveAddress
transferOwnership
<Constructor>
recoverERC20
startVesting
address
VM888Vesting.FutureReserveAlreadySet
VM888Vesting.ZeroAddress
tokenAddress
call
IVM888Token (iface)
balanceOf
transfer
getCurrentPrice
getEMA

Summary

VaultedMetrics contract implements a vesting mechanism. This audit investigates security issues, business logic concerns and potential improvements. VaultedMetrics is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler error or critical issues.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io